

Manipal School of Information Sciences

Linux Operating System Lab Assignment

AWK Pattern-Action Programming Lab

Employee Records (D1) and Security Event Log (D2)

Student Name:	Ribhav Devprasad
Reg. No.:	26110690002
Date:	3 November 2026

AWK Lab: Patterns, Actions and Control Flow

Objective and setup

This lab uses `awk` to select records with patterns (D1, `employee.txt`) and to write complete awk programs with variables, `printf`, arrays, conditionals and loops (D2, `security_log.csv`). Every command was executed and the output shown is real. The input files are only read, never modified. The supplied files had Windows line endings, so they were converted with `sed -i 's/\r$//'` first.

D1 input: employee.txt

Fields: `$1` Employee ID, `$2` name, `$3` department, `$4` salary, `$5` status.

```
$ cat employee.txt
101 Alice Security 72000 Active
102 Bob VLSI 65000 Active
103 Carol Security 48000 Inactive
104 David VLSI 82000 Active
105 Eva Networking 55000 Active
106 Frank Security 90000 Active
107 Grace VLSI 45000 Inactive
108 Henry Networking 76000 Active
109 Irene Security 68000 Active
110 John VLSI 78000 Active
111 Kavya Networking 42000 Inactive
112 Liam Security 85000 Active
```

D2 input: security_log.csv

Comma-separated; fields: `$1` Time, `$2` Source_IP, `$3` Event, `$4` Severity, `$5` Attempts. The first line is a header, so programs use `NR>1` to skip it.

```
$ cat security_log.csv
Time,Source_IP,Event,Severity,Attempts
09:10,192.168.1.10,Login_Success,LOW,1
09:12,192.168.1.25,Login_Failed,MEDIUM,3
09:15,10.0.0.8,Port_Scan,HIGH,12
09:18,192.168.1.25,Login_Failed,HIGH,7
09:22,10.0.0.12,Malware_Alert,CRITICAL,1
09:25,192.168.1.30,Login_Success,LOW,1
```

D1: Pattern blocks on employee.txt

Part A: Numerical and string comparisons

A1. Name, department and salary of employees earning more than 70000

```
$ awk '$4 > 70000 {print $2, $3, $4}' employee.txt
Alice Security 72000
David VLSI 82000
```

```
Frank Security 90000
Henry Networking 76000
John VLSI 78000
Liam Security 85000
```

A2. Salary between 60000 and 80000 inclusive

```
$ awk '$4 >= 60000 && $4 <= 80000' employee.txt
101 Alice Security 72000 Active
102 Bob VLSI 65000 Active
108 Henry Networking 76000 Active
109 Irene Security 68000 Active
110 John VLSI 78000 Active
```

A3. Employment status Inactive

```
$ awk '$5 == "Inactive"' employee.txt
103 Carol Security 48000 Inactive
107 Grace VLSI 45000 Inactive
111 Kavya Networking 42000 Inactive
```

A4. Employees in the VLSI department

```
$ awk '$3 == "VLSI"' employee.txt
102 Bob VLSI 65000 Active
104 David VLSI 82000 Active
107 Grace VLSI 45000 Inactive
110 John VLSI 78000 Active
```

Part B: Regular-expression patterns

B1. All records containing the word Security

```
$ awk '/Security/' employee.txt
101 Alice Security 72000 Active
103 Carol Security 48000 Inactive
106 Frank Security 90000 Active
109 Irene Security 68000 Active
112 Liam Security 85000 Active
```

B2. Names beginning with A, B or C (regex applied only to the name field)

```
$ awk '$2 ~ /^[ABC]/' employee.txt
101 Alice Security 72000 Active
102 Bob VLSI 65000 Active
103 Carol Security 48000 Inactive
```

Using `$2 ~` restricts the match to the name field; a bare `/^[ABC]/` would test the whole record, which starts with the ID digits.

B3. Department field does not match Security

```
$ awk '$3 !~ /Security/' employee.txt
102 Bob VLSI 65000 Active
104 David VLSI 82000 Active
105 Eva Networking 55000 Active
107 Grace VLSI 45000 Inactive
108 Henry Networking 76000 Active
```

```
110 John VLSI 78000 Active
111 Kavya Networking 42000 Inactive
```

Part C: Logical patterns

C1. Security department and salary above 70000

```
$ awk '$3 == "Security" && $4 > 70000' employee.txt
101 Alice Security 72000 Active
106 Frank Security 90000 Active
112 Liam Security 85000 Active
```

C2. VLSI or Networking department

```
$ awk '$3 == "VLSI" || $3 == "Networking"' employee.txt
102 Bob VLSI 65000 Active
104 David VLSI 82000 Active
105 Eva Networking 55000 Active
107 Grace VLSI 45000 Inactive
108 Henry Networking 76000 Active
110 John VLSI 78000 Active
111 Kavya Networking 42000 Inactive
```

C3. VLSI, salary above 70000 and Active

```
$ awk '$3 == "VLSI" && $4 > 70000 && $5 == "Active"' employee.txt
104 David VLSI 82000 Active
110 John VLSI 78000 Active
```

Part D: Conditional pattern

D1. Security: salary above 75000; otherwise above 70000 (name, department, salary)

```
$ awk '$4 > ($3 == "Security" ? 75000 : 70000) {print $2, $3, $4}' employee.txt
David VLSI 82000
Frank Security 90000
Henry Networking 76000
John VLSI 78000
Liam Security 85000
```

The conditional operator chooses the threshold per record, and the pattern compares the salary with it.

Part E: Grouped patterns

E1. Active employees in Security or VLSI (Active applies to both)

```
$ awk '$5 == "Active" && ($3 == "Security" || $3 == "VLSI")' employee.txt
101 Alice Security 72000 Active
102 Bob VLSI 65000 Active
104 David VLSI 82000 Active
106 Frank Security 90000 Active
109 Irene Security 68000 Active
110 John VLSI 78000 Active
112 Liam Security 85000 Active
```

Parentheses are essential: without them `&&` binds tighter than `||` and Active would apply only to VLSI.

E2. (Security and salary above 80000) or (VLSI and salary above 75000)

```
$ awk '($3 == "Security" && $4 > 80000) || ($3 == "VLSI" && $4 > 75000)' employee.txt
104 David VLSI 82000 Active
106 Frank Security 90000 Active
110 John VLSI 78000 Active
112 Liam Security 85000 Active
```

E3. Active employees satisfying (Security and above 70000) or (VLSI and above 75000)

```
$ awk '$5 == "Active" && (($3 == "Security" && $4 > 70000) || ($3 == "VLSI" && $4 > 75000))'
employee.txt
101 Alice Security 72000 Active
104 David VLSI 82000 Active
106 Frank Security 90000 Active
110 John VLSI 78000 Active
112 Liam Security 85000 Active
```

Part F: Negated patterns

F1. Status is not Active (exact comparison, because Inactive contains the text Active)

```
$ awk '$5 != "Active"' employee.txt
103 Carol Security 48000 Inactive
107 Grace VLSI 45000 Inactive
111 Kavya Networking 42000 Inactive
```

A regex such as `!/Active/` would wrongly drop `Inactive` records as well, since they contain the substring. Comparing the whole field avoids that.

F2. Neither Security nor VLSI

```
$ awk '!($3 == "Security" || $3 == "VLSI")' employee.txt
105 Eva Networking 55000 Active
108 Henry Networking 76000 Active
111 Kavya Networking 42000 Inactive
```

Part G: Range patterns

G1. From employee ID 103 to employee ID 108 (inclusive)

```
$ awk '$1 == 103, $1 == 108' employee.txt
103 Carol Security 48000 Inactive
104 David VLSI 82000 Active
105 Eva Networking 55000 Active
106 Frank Security 90000 Active
107 Grace VLSI 45000 Inactive
108 Henry Networking 76000 Active
```

G2. From David to Irene (inclusive)

```
$ awk '$2 == "David", $2 == "Irene"' employee.txt
104 David VLSI 82000 Active
105 Eva Networking 55000 Active
106 Frank Security 90000 Active
107 Grace VLSI 45000 Inactive
```

```
108 Henry Networking 76000 Active
109 Irene Security 68000 Active
```

Part H: Built-in variables

H1. First five records with record number

```
$ awk 'NR <= 5 {print NR, $0}' employee.txt
1 101 Alice Security 72000 Active
2 102 Bob VLSI 65000 Active
3 103 Carol Security 48000 Inactive
4 104 David VLSI 82000 Active
5 105 Eva Networking 55000 Active
```

H2. Records 4 to 9 using NR

```
$ awk 'NR >= 4 && NR <= 9' employee.txt
104 David VLSI 82000 Active
105 Eva Networking 55000 Active
106 Frank Security 90000 Active
107 Grace VLSI 45000 Inactive
108 Henry Networking 76000 Active
109 Irene Security 68000 Active
```

H3. Records with exactly five fields (record number and record)

```
$ awk 'NF == 5 {print NR, $0}' employee.txt
1 101 Alice Security 72000 Active
2 102 Bob VLSI 65000 Active
3 103 Carol Security 48000 Inactive
4 104 David VLSI 82000 Active
5 105 Eva Networking 55000 Active
6 106 Frank Security 90000 Active
7 107 Grace VLSI 45000 Inactive
8 108 Henry Networking 76000 Active
9 109 Irene Security 68000 Active
10 110 John VLSI 78000 Active
11 111 Kavya Networking 42000 Inactive
12 112 Liam Security 85000 Active
```

H4. Odd-numbered records

```
$ awk 'NR % 2 == 1' employee.txt
101 Alice Security 72000 Active
103 Carol Security 48000 Inactive
105 Eva Networking 55000 Active
107 Grace VLSI 45000 Inactive
109 Irene Security 68000 Active
111 Kavya Networking 42000 Inactive
```

Part I: Arithmetic conditions

I1. Bonus = 10% of salary; employees whose bonus exceeds 7000 (name, salary, bonus)

```
$ awk '$4 * 0.10 > 7000 {print $2, $4, $4 * 0.10}' employee.txt
Alice 72000 7200
David 82000 8200
Frank 90000 9000
Henry 76000 7600
```

```
John 78000 7800
Liam 85000 8500
```

I2. Revised salary after a 12% increment; show those above 85000

```
$ awk '$4 * 1.12 > 85000 {print $2, $4, $4 * 1.12}' employee.txt
David 82000 91840
Frank 90000 100800
Henry 76000 85120
John 78000 87360
Liam 85000 95200
```

I3. Monthly salary (annual / 12) above 6000

```
$ awk '$4 / 12 > 6000 {print $2, $4, $4 / 12}' employee.txt
David 82000 6833.33
Frank 90000 7500
Henry 76000 6333.33
John 78000 6500
Liam 85000 7083.33
```

Part J: Integrated challenge

Report of Active employees who satisfy one of: Security with salary of at least 70000, VLSI with at least 75000, Networking with at least 70000. `BEGIN` prints the heading and `END` prints the total.

```
$ awk 'BEGIN { print "ACTIVE EMPLOYEE REPORT"
               printf "%-4s %-8s %-12s %-8s %s\n", "ID", "Name", "Department", "Salary",
"Status" }
     $5 == "Active" && (($3 == "Security"   && $4 >= 70000) ||
                        ($3 == "VLSI"       && $4 >= 75000) ||
                        ($3 == "Networking" && $4 >= 70000)) {
         printf "%-4s %-8s %-12s %-8s %s\n", $1, $2, $3, $4, $5
         n++ }
     END { print "Total selected employees:", n + 0 }' employee.txt
ACTIVE EMPLOYEE REPORT
ID   Name     Department   Salary   Status
101  Alice    Security     72000    Active
104  David    VLSI         82000    Active
106  Frank    Security     90000    Active
108  Henry    Networking   76000    Active
110  John     VLSI         78000    Active
112  Liam     Security     85000    Active
Total selected employees: 6
```

D2: AWK programs on security_log.csv

1. Source IP and event of every record

```
$ awk -F, 'NR > 1 {print $2, $3}' security_log.csv
192.168.1.10 Login_Success
192.168.1.25 Login_Failed
10.0.0.8 Port_Scan
192.168.1.25 Login_Failed
10.0.0.12 Malware_Alert
192.168.1.30 Login_Success
```

2. Time, source IP and attempts in aligned columns (printf)

```
$
awk -F, 'NR == 1 {next} NR == 2 {printf "%-6s %-15s %8s\n", "Time", "Source_IP", "Attempts"}
{printf "%-6s %-15s %8d\n", $1, $2, $5}' security_log.csv
Time   Source_IP       Attempts
09:10  192.168.1.10           1
09:12  192.168.1.25           3
09:15  10.0.0.8              12
09:18  192.168.1.25           7
09:22  10.0.0.12              1
09:25  192.168.1.30           1
```

3. Store source IP and event in variables and print a sentence

```
$ awk -F, 'NR > 1 {ip = $2; ev = $3; print "Source " ip " generated event " ev "."}'
security_log.csv
Source 192.168.1.10 generated event Login_Success.
Source 192.168.1.25 generated event Login_Failed.
Source 10.0.0.8 generated event Port_Scan.
Source 192.168.1.25 generated event Login_Failed.
Source 10.0.0.12 generated event Malware_Alert.
Source 192.168.1.30 generated event Login_Success.
```

4. Processing time per record at 2 seconds per attempt

```
$ awk -F, 'NR > 1 {t = $5 * 2; print $2, $3, t " seconds"}' security_log.csv
192.168.1.10 Login_Success 2 seconds
192.168.1.25 Login_Failed 6 seconds
10.0.0.8 Port_Scan 24 seconds
192.168.1.25 Login_Failed 14 seconds
10.0.0.12 Malware_Alert 2 seconds
192.168.1.30 Login_Success 2 seconds
```

5. Total number of attempts

```
$ awk -F, 'NR > 1 {total += $5} END {print "Total attempts:", total}' security_log.csv
Total attempts: 25
```

6. Average attempts per record

```
$ awk -F, 'NR > 1 {total += $5; n++} END {printf "Average attempts per record: %.2f\n",
total / n}' security_log.csv
Average attempts per record: 4.17
```

7. Number of Login_Failed records

```
$ awk -F, '$3 == "Login_Failed" {c++} END {print "Login_Failed records:", c + 0}'
security_log.csv
Login_Failed records: 2
```

8. Review quota of 10, minus one for each HIGH or CRITICAL record

```
$
awk -F, 'BEGIN {quota = 10} $4 == "HIGH" || $4 == "CRITICAL" {quota--} END {print "Remaining
quota:", quota}' security_log.csv
Remaining quota: 7
```

9. Replace Port_Scan with Network_Scan and print every record as CSV

```
$ awk 'BEGIN {FS = OFS = ","} $3 == "Port_Scan" {$3 = "Network_Scan"} {print}'
security_log.csv
Time,Source_IP,Event,Severity,Attempts
09:10,192.168.1.10,Login_Success,LOW,1
09:12,192.168.1.25,Login_Failed,MEDIUM,3
09:15,10.0.0.8,Network_Scan,HIGH,12
09:18,192.168.1.25,Login_Failed,HIGH,7
09:22,10.0.0.12,Malware_Alert,CRITICAL,1
09:25,192.168.1.30,Login_Success,LOW,1
```

Setting `OFS` to a comma makes awk rebuild the record with commas after a field is changed. Unmodified records print unchanged.

10. Add one to attempts for each Login_Failed record, print every record as CSV

```
$ awk 'BEGIN {FS = OFS = ","} $3 == "Login_Failed" {$5 = $5 + 1} {print}' security_log.csv
Time,Source_IP,Event,Severity,Attempts
09:10,192.168.1.10,Login_Success,LOW,1
09:12,192.168.1.25,Login_Failed,MEDIUM,4
09:15,10.0.0.8,Port_Scan,HIGH,12
09:18,192.168.1.25,Login_Failed,HIGH,8
09:22,10.0.0.12,Malware_Alert,CRITICAL,1
09:25,192.168.1.30,Login_Success,LOW,1
```

11. Source IP and event when severity is HIGH and attempts exceed 5

```
$ awk -F, '$4 == "HIGH" && $5 > 5 {print $2, $3}' security_log.csv
10.0.0.8 Port_Scan
192.168.1.25 Login_Failed
```

12. Source IP and event when severity is HIGH or CRITICAL

```
$ awk -F, '$4 == "HIGH" || $4 == "CRITICAL" {print $2, $3}' security_log.csv
10.0.0.8 Port_Scan
192.168.1.25 Login_Failed
10.0.0.12 Malware_Alert
```

13. Conditional operator: Repeated if attempts exceed 1, else Single

```
$ awk -F, 'NR > 1 {label = ($5 > 1) ? "Repeated" : "Single"; print $2, $5, label}'
security_log.csv
192.168.1.10 1 Single
192.168.1.25 3 Repeated
10.0.0.8 12 Repeated
192.168.1.25 7 Repeated
10.0.0.12 1 Single
192.168.1.30 1 Single
```

14. Conditional operator: Investigate if attempts are at least 5, else Routine

```
$ awk -F, 'NR > 1 {label = ($5 >= 5) ? "Investigate" : "Routine"; print $2, $5, label}'
security_log.csv
192.168.1.10 1 Routine
192.168.1.25 3 Routine
10.0.0.8 12 Investigate
192.168.1.25 7 Investigate
```

```
10.0.0.12 1 Routine
192.168.1.30 1 Routine
```

15. if-else: Review required for HIGH or CRITICAL, otherwise Routine review

```
$
awk -F, 'NR > 1 {if ($4 == "HIGH" || $4 == "CRITICAL") print $2, $4, "Review required"; else
print $2, $4, "Routine review"}' security_log.csv
192.168.1.10 LOW Routine review
192.168.1.25 MEDIUM Routine review
10.0.0.8 HIGH Review required
192.168.1.25 HIGH Review required
10.0.0.12 CRITICAL Review required
192.168.1.30 LOW Routine review
```

16. if / else if / else: activity level by attempts

```
$ awk -F, 'NR > 1 {if ($5 >= 10) lvl = "High activity"; else if ($5 >= 5) lvl = "Moderate
activity"; else lvl = "Low activity"; print $2, $5, lvl}' security_log.csv
192.168.1.10 1 Low activity
192.168.1.25 3 Low activity
10.0.0.8 12 High activity
192.168.1.25 7 Moderate activity
10.0.0.12 1 Low activity
192.168.1.30 1 Low activity
```

17. for loop: Check 1 to Check 5

```
$ awk 'BEGIN {for (i = 1; i <= 5; i++) print "Check", i}'
Check 1
Check 2
Check 3
Check 4
Check 5
```

18. while loop: count down from 10 to 1

```
$ awk 'BEGIN {i = 10; while (i >= 1) {printf "%d ", i; i--}; print ""}'
10 9 8 7 6 5 4 3 2 1
```

19. do-while loop: count down from 10 to 1

```
$ awk 'BEGIN {i = 10; do {printf "%d ", i; i--} while (i >= 1); print ""}'
10 9 8 7 6 5 4 3 2 1
```

20. Aligned report with activity classification and END totals

```
$ awk -F, 'NR == 1 {next}
     NR == 2 {printf "%-15s %-15s %-9s %s\n", "Source_IP", "Event", "Attempts", "Activity"}
     {   if ($5 >= 10) lvl = "High"; else if ($5 >= 5) lvl = "Moderate"; else lvl = "Low"
         printf "%-15s %-15s %-9d %s\n", $2, $3, $5, lvl
         records++; total += $5 }
     END {print "Total records:", records; print "Total attempts:", total}' security_log.csv
Source_IP       Event           Attempts  Activity
192.168.1.10    Login_Success   1         Low
192.168.1.25    Login_Failed    3         Low
10.0.0.8        Port_Scan       12        High
192.168.1.25    Login_Failed    7         Moderate
10.0.0.12       Malware_Alert   1         Low
192.168.1.30    Login_Success   1         Low
```

```
Total records: 6
Total attempts: 25
```

Scenario tasks (saved as separate .awk script files)

Login monitoring

login_monitor.awk

```
# Login monitoring: Login_Failed entries with 5 or more attempts are suspicious
BEGIN { FS = ","; print "Suspicious failed-login entries (attempts >= 5)" }
NR > 1 && $3 == "Login_Failed" && $5 >= 5 {
    printf "%-15s %d\n", $2, $5
    suspicious++
}
END { print "Suspicious entries found:", suspicious + 0 }
```

```
$ awk -f login_monitor.awk security_log.csv
Suspicious failed-login entries (attempts >= 5)
192.168.1.25    7
Suspicious entries found: 1
```

Incident summary

incident_summary.awk

```
# Incident summary: entries per severity category
BEGIN { FS = "," }
NR > 1 { count[$4]++ }
END {
    print "Incident summary by severity"
    n = split("LOW MEDIUM HIGH CRITICAL", order, " ")
    for (i = 1; i <= n; i++)
        printf "%-9s %d\n", order[i], count[order[i]] + 0
}
```

```
$ awk -f incident_summary.awk security_log.csv
Incident summary by severity
LOW       2
MEDIUM    1
HIGH      2
CRITICAL  1
```

Processing estimate

processing_estimate.awk

```
# Processing estimate: 2 seconds of analyst review per attempt
BEGIN { FS = ","; printf "%-6s %-15s %8s %12s\n", "Time", "Source_IP", "Attempts", "Review
(s)" }
NR > 1 {
    secs = $5 * 2
    total += secs
    printf "%-6s %-15s %8d %12d\n", $1, $2, $5, secs
}
END { printf "Total review time: %d seconds (%.2f minutes)\n", total, total / 60 }
```

```
$ awk -f processing_estimate.awk security_log.csv
Time   Source_IP       Attempts   Review (s)
09:10  192.168.1.10           1            2
```

```
09:12  192.168.1.25           3            6
09:15  10.0.0.8               12           24
09:18  192.168.1.25           7            14
09:22  10.0.0.12              1            2
09:25  192.168.1.30           1            2
Total review time: 50 seconds (0.83 minutes)
```

Log normalisation

normalise_log.awk

```
# Log normalisation: Login_Failed -> Failed_Login, output as CSV (input is never modified)
BEGIN { FS = OFS = "," }
$3 == "Login_Failed" { $3 = "Failed_Login" }
{ print }
```

```
$ awk -f normalise_log.awk security_log.csv > normalised_log.csv && cat normalised_log.csv
Time,Source_IP,Event,Severity,Attempts
09:10,192.168.1.10,Login_Success,LOW,1
09:12,192.168.1.25,Failed_Login,MEDIUM,3
09:15,10.0.0.8,Port_Scan,HIGH,12
09:18,192.168.1.25,Failed_Login,HIGH,7
09:22,10.0.0.12,Malware_Alert,CRITICAL,1
09:25,192.168.1.30,Login_Success,LOW,1
```

Priority queue

priority_queue.awk

```
# Priority queue: CRITICAL -> Immediate, HIGH -> Soon, everything else -> Routine
BEGIN { FS = ","; printf "%-6s %-15s %-15s %s\n", "Time", "Source_IP", "Event", "Priority" }
NR > 1 {
    if ($4 == "CRITICAL")  p = "Immediate"
    else if ($4 == "HIGH") p = "Soon"
    else                   p = "Routine"
    printf "%-6s %-15s %-15s %s\n", $1, $2, $3, p
}
```

```
$ awk -f priority_queue.awk security_log.csv
Time   Source_IP       Event           Priority
09:10  192.168.1.10    Login_Success   Routine
09:12  192.168.1.25    Login_Failed    Routine
09:15  10.0.0.8        Port_Scan       Soon
09:18  192.168.1.25    Login_Failed    Soon
09:22  10.0.0.12       Malware_Alert   Immediate
09:25  192.168.1.30    Login_Success   Routine
```

```
$ grep -c Login_Failed security_log.csv   # source file still contains the original event name
0d77042239d4e2daf83e8a5ef791e097  security_log.csv
2
```

Summary of awk features used

Feature	Used for
Pattern `cond {action}`	Run the action only for records where the condition is true
`~`, `!~`, `/re/`	Regular-expression match on a field or the whole record

`&&`, `||`, `!`, parentheses	Combine and group conditions
Range `p1, p2`	From the first record matching p1 to the next matching p2, inclusive
`NR`, `NF`, `FS`, `OFS`	Record number, field count, input and output separators
`BEGIN` / `END`	Run once before the first record / after the last record
Arrays, `split`	Counting events per category
`?:`, `if-else`, `for`, `while`, `do-while`	Conditional logic and loops
`printf`	Aligned, formatted output